Enterprise AI Workflow & Automation Platform

Interview Preparation — 60 Questions & Answers

Covers architecture, RabbitMQ, Celery, Redis, PostgreSQL, the API Gateway, LangGraph/LLM systems, human-in-the-loop design, idempotency, correlation IDs, security, testing, and Docker/local deployment — organized the way an interviewer is likely to move through the topics.

Each answer is written to be spoken in under 20 seconds — enough to show understanding without over-explaining. Use it as a lead-in; be ready for a natural follow-up question on anything you say.

Architecture & Microservices

Q1. Why did you split this into microservices instead of one FastAPI app?

To demonstrate independent deployability, failure isolation, and per-service scaling — a monolith would be faster to build but wouldn't exercise service boundaries, which was the explicit point of this portfolio project.

Q2. How many services does the platform have, and what does each own?

7 services: api-gateway (routing/auth), auth-service (users/JWT), workflow-service (state machine), ai-service (LLM reasoning), integration-service (mock external APIs), notification-service (delivery), audit-service (event log). Each owns its own data.

Q3. When would you NOT use microservices?

When the team is small, the domain is simple, or you don't yet know the real service boundaries — a modular monolith is faster to build and refactor; splitting too early adds network calls and operational overhead for no real benefit.

Q4. How do services communicate with each other?

Synchronous HTTP for anything needing an immediate answer (Gateway to Auth, AI Service calling Integration Service's tool lookups), and asynchronous Celery/RabbitMQ dispatch for long-running or decoupled work like the AI/integration/notification steps.

Q5. What's in shared/, and why is it kept small?

Only infrastructure plumbing — logging, JWT helpers, event contracts, messaging utilities. No business logic. If workflow logic lived there, every service would be silently coupled to it, turning the system into a distributed monolith.

Q6. How do you prevent services from reaching into each other's databases?

Each service owns its own Postgres schema and only exposes data through its own HTTP API. No service's SQLAlchemy models reference another service's tables directly — enforced by convention and code review, not the database engine.

Q7. What's the API Gateway's job, and what does it deliberately NOT do?

It's the single public entry point: JWT verification, rate limiting, routing, correlation IDs, CORS, unified error format. It does not contain business logic or touch a database directly.

Q8. Why does the Gateway check authentication but services still check authorization?

The Gateway verifying the JWT once avoids a network call per request, but each service independently re-checks permissions for its own sensitive operations — defense in depth, so a bug in the Gateway's routing can't silently bypass a service's own access control.

Event-Driven Architecture & RabbitMQ

Q9. Why RabbitMQ instead of Kafka?

This system needs per-message routing, retries, and dead-lettering, which RabbitMQ's exchange/routing-key/ack model fits naturally. Kafka is stronger for high-throughput streaming and replay, which isn't the need here.

Q10. Explain the exchange → routing key → queue → consumer flow in this project.

Audit events publish to a topic exchange (events.exchange) with a routing key like workflow.completed; the audit queue is bound with a catch-all # key so it receives every event type and persists it.

Q11. What does an event look like in this system?

A structured EventEnvelope with event_id, event_type, timestamp, correlation_id, workflow_id, execution_id, and a payload — never an arbitrary unstructured message, so every consumer can rely on the same shape.

Q12. How do you guarantee a RabbitMQ consumer doesn't lose a message if it crashes mid-processing?

Late acknowledgement (task_acks_late=True) — the broker only removes the message once processing actually finishes, so a crash mid-task leaves it unacked and redeliverable to another worker.

Q13. How does your dead-letter queue (DLQ) actually work?

Each processing queue is declared with an x-dead-letter-exchange; when a task exhausts retries, the worker re-raises instead of swallowing the error, so the broker doesn't ack it and routes it to a <queue>.dlq for operator inspection.

Q14. Walk me through your audit consumer's retry cascade.

On failure it publishes the message to a retry queue with a per-message TTL (5s/15s/30s depending on attempt count); when the TTL expires, RabbitMQ dead-letters it back into the main exchange for re-delivery — after 3 attempts, it goes straight to the DLQ instead.

Q15. Why do you have two different retry mechanisms in one project?

Celery already is a retry mechanism, so the three Celery-backed queues use Celery's own retry; the audit consumer has no task framework underneath it, so it needs the raw RabbitMQ TTL-cascade pattern. Using the same mechanism everywhere 'for consistency' would mean picking the worse-fitting tool in one of the two places.

Q16. What's the difference between at-least-once delivery and exactly-once processing?

RabbitMQ guarantees at-least-once — a message can be redelivered after a crash before ack. Exactly-once processing is achieved at the application layer via idempotency (unique event_id with ON CONFLICT DO NOTHING), not by the broker itself.

Celery & Background Processing

Q17. Why Celery instead of FastAPI's BackgroundTasks?

BackgroundTasks run in-process and die with it — no retry, no independent scaling. Celery workers survive a process restart, retry with backoff, and scale horizontally independent of the API servers.

Q18. How do retries work for a failed AI or integration call?

A fixed 5s/15s/30s schedule via self.retry(countdown=...), but only for retryable errors — network failures and 5xx responses. A 4xx fails immediately since retrying it would never succeed.

Q19. Why don't you retry a 404 from Integration Service?

A 404 means the resource genuinely doesn't exist — retrying wastes up to 3 attempts plus backoff time for zero chance of success, and for a side-effecting call like a refund, unnecessary retries are pure added risk for no benefit.

Q20. How is a refund call protected from being executed twice by a retry?

The integration worker derives a stable Idempotency-Key from execution_id + action (not regenerated per attempt), so even if a retry fires after the first call actually succeeded server-side, Integration Service returns the cached result instead of refunding again.

Q21. What does task_acks_late actually buy you?

Crash-safety: the message isn't removed from the queue until the task finishes successfully, so a worker crash mid-task means another worker picks the same message back up instead of it being silently lost.

Q22. How do you prevent a worker from endlessly retrying forever?

A fixed maximum retry count (len(RETRY_DELAYS) = 3) tracked via next_retry_delay(); once exhausted, the worker reports failure to Workflow Service and re-raises so the message dead-letters instead of retrying indefinitely.

Redis

Q23. What is Redis actually used for in this project?

Distributed rate limiting at the Gateway (fixed-window counters) and as Celery's result backend. It is never the system of record for business data — Postgres owns that.

Q24. Why fixed-window rate limiting instead of a sliding window?

Simplicity and fewer Redis calls per request; the known trade-off is bursts at window boundaries, which a sliding-window-log or token-bucket would smooth out at higher Redis cost — acceptable for this scale.

PostgreSQL & Data Design

Q25. Why schema-per-service instead of separate database instances?

Logical isolation without running 4-5x the Postgres containers locally — a deliberate trade-off for a portfolio project; true physical isolation via separate instances is the documented next step for real production.

Q26. How do you keep one service from querying another's tables?

Discipline plus boundaries: no service's SQLAlchemy models ever reference another service's schema; any cross-service data need goes through that service's HTTP API instead.

Q27. Why does only one service (workflow-service) use real Alembic migrations?

It's the reference implementation of the pattern; the other three stateful services still use create_all() at startup, which works for a fresh database but can't evolve an existing one — a documented, honest gap, not hidden.

Q28. Walk me through the idempotency implementation for workflow execution.

WorkflowExecution has a unique (workflow_id, idempotency_key) constraint. If a client resends the same Idempotency-Key header, the endpoint returns the existing execution instead of creating a second one.

Q29. What indexes did you actually add, and why?

Fields used in lookups and joins: execution_id, correlation_id, event_id, jti — based on actual query patterns, not indexing every column, which would slow writes for no read benefit.

AI / LLM / LangGraph

Q30. Why LangGraph only for the AI reasoning flow and not the whole workflow engine?

The AI path (intent → plan → tool selection → validate → decide) has real branching that benefits from an explicit graph; the deterministic workflow state machine doesn't need an AI framework and would just add an unnecessary dependency.

Q31. Describe your LLM provider abstraction.

An LLMProvider interface with one method, generate_structured(); Mock/OpenAI/Groq/Gemini implementations plug in behind it, selected by one environment variable — switching providers never touches the reasoning flow or routes.

Q32. Why does a Mock provider exist, and why is it the default?

It runs the full reasoning flow deterministically with zero cost and no API key, which is what lets tests and the eval harness run reliably in CI without depending on a live, variable LLM.

Q33. How do you validate that the LLM's output is actually usable?

A Pydantic schema (DecisionOutput) validates the required fields and types; invalid output gets one repair-retry with an explicit correction prompt, and if it's still invalid, the step fails safe rather than acting on unvalidated data.

Q34. What tools can the AI actually call, and how are they controlled?

Only explicitly registered, Pydantic-validated tools — customer/order/ticket lookup — proxied through Integration Service. The LLM never gets direct database or arbitrary HTTP access.

Q35. Why can't the AI directly execute a refund?

Side-effecting actions must go through Workflow Service's approval gate; the AI only produces a recommendation. This is enforced structurally — Integration Service is only ever called by a worker acting on an explicit dispatch, never by AI Service.

Q36. What happens when a tool call fails, like a lookup for a nonexistent order?

The decision node returns recommended_action: 'fail', and the orchestrator marks the execution FAILED — not WAITING_FOR_APPROVAL, since asking a human to approve a decision built on missing data would be the wrong default.

Q37. What does your AI evaluation harness actually measure?

Intent-classification accuracy, structured-output validity, approval-gate accuracy, invalid-tool-call rate, latency, and token/cost — run against 100 synthetic requests, reproducibly, against the Mock provider by default.

Human-in-the-Loop & AI Safety

Q38. How does the human approval gate work end to end?

AI Service flags requires_approval based on risk; Workflow Service creates an Approval row and moves the execution to WAITING_FOR_APPROVAL. Only an explicit approve call dispatches the side-effecting action — reject cancels it instead.

Q39. What decides whether something needs approval?

A risk heuristic in the decision step — for example, a refund amount at or above a threshold is flagged high-risk and requires_approval is set to true; low-risk decisions proceed automatically.

Q40. How do you stop a prompt-injected or hallucinating LLM from bypassing approval?

Structurally, not by trusting the model: Integration Service's side-effecting endpoints are only ever called by a worker acting on Workflow Service's explicit dispatch after approval — the AI has no code path that reaches them directly.

Q41. What AI safety practices does this project actually implement, concretely?

Structured-output validation with repair-retry, explicitly registered and Pydantic-validated tools only, no direct DB/HTTP access for the LLM, and a human approval gate before any side-effecting action.

Idempotency & Correlation IDs

Q42. Where is idempotency implemented in this system, and why in those specific places?

On workflow execution (Idempotency-Key header, unique per workflow) and on the refund call (Idempotency-Key derived from execution_id+action) — the two places a retried or duplicated request could cause a real, unwanted side effect.

Q43. How do you make an event consumer idempotent against duplicate delivery?

A unique constraint on event_id plus ON CONFLICT DO NOTHING on insert — a redelivered message is a no-op the second time, which is what 'at-least-once delivery, idempotent consumer' means in practice.

Q44. What is a correlation ID, and how does it differ from a request ID?

A request ID is per-HTTP-call; a correlation ID is per-logical-operation and is propagated across every hop (Gateway, services, Celery task kwargs, worker callbacks) so one workflow execution can be traced end to end in the logs.

Q45. How would you trace one failed workflow execution across all 7 services?

Query Audit Service for events filtered by correlation_id — every event emitted during that execution carries the same correlation_id, giving a complete, ordered cross-service trail without grepping individual service logs.

Security & Auth

Q46. Describe your authentication flow.

JWT access + refresh tokens issued by Auth Service; the Gateway verifies the access token locally using a shared secret (avoiding a network hop per request) and forwards the decoded identity downstream.

Q47. How does refresh token rotation work, and why does it matter?

Each refresh call issues a new refresh token and revokes the old one; reusing an already-rotated token fails. This limits the damage window if a refresh token is ever leaked.

Q48. How is RBAC implemented?

Four roles (ADMIN, OPERATOR, APPROVER, USER) map to explicit permission strings like workflow:execute or audit:read in a central roles map; routes declare the permission they require via a dependency.

Q49. Why does the Gateway verify JWTs locally instead of calling Auth Service every time?

Latency — a network round-trip on every single request would add cost and a new failure mode (Auth Service down taking down everything). A shared signing secret lets any service verify a token's validity without asking Auth Service.

Q50. What would you change to harden this for real production use?

Short-lived access tokens with a key-rotation strategy instead of a static shared secret, mutual TLS or a signed service-identity header between internal services, and moving from create_all() to Alembic everywhere.

Q51. How is sensitive data kept out of logs?

Structured logging explicitly avoids logging passwords, JWTs, and API keys by convention in every log call — the logger itself doesn't auto-redact, since a missed field name in a regex gives false confidence; discipline is more reliable here.

Observability & Testing

Q52. What's the difference between your /health and /ready endpoints?

/health means the process is alive — no dependency checks, ever. /ready means it can currently serve requests — cheap, timeout-bounded checks against Postgres/Redis/RabbitMQ, so a probe hitting it every few seconds stays cheap.

Q53. What does a structured log line actually contain?

timestamp, service, environment, level, message, request_id, correlation_id, and any extra context like execution_id or latency — one JSON object per line, directly queryable in any log aggregator without custom parsing.

Q54. How many automated tests does this project have, and how are they organized?

46 tests, one suite per service plus a shared suite for pure logic like the retry policy — each runs in isolation with its own PYTHONPATH, avoiding multiple services' same-named 'app' package colliding in one pytest session.

Q55. Why do E2E tests live separately and self-skip?

They hit the real running stack over HTTP with nothing mocked, specifically to catch contract mismatches between services that per-service unit tests can't see — but that means they need the whole stack running, so they skip gracefully rather than failing CI when it isn't up.

Q56. What's a bug your tests actually caught, not just a theoretical one?

A naive/aware datetime mismatch in a duration calculation — SQLite round-trips timestamps as naive while Postgres preserves timezone info, and the orchestrator test caught the crash before it ever reached production code.

Docker, Local Dev & Deployment

Q57. Why does every service get its own Dockerfile instead of one shared image?

Independent deployability — each service has its own dependencies and can be rebuilt, scaled, or redeployed without touching the others, which is the actual point a multi-container setup is supposed to demonstrate.

Q58. How do containers find each other?

By Docker service name over the compose network (http\://auth-service:8001), never localhost — localhost inside a container refers to that container itself, not its neighbors.

Q59. How did you support running the project without Docker at all?

A shared Postgres/SQLite compatibility layer that strips schema qualifiers for SQLite, routed through the exact same init_models() call every service already used — Docker/Postgres mode is completely untouched by the change.

Q60. What's the trade-off of the no-Docker local mode?

SQLite instead of Postgres (no schema-per-service isolation, no Alembic for workflow-service in that mode) — real RabbitMQ and Redis are still required, since faking those away would mean you're no longer testing the actual event-driven architecture.